set_pg_strategy_via_rule

NAME

set_pg_strategy_via_rule
Specifies the via rule between strategies and collections of existing shapes.

SYNTAX

```
status set_pg_strategy_via_rule
       rule_name
       -via_rule via_rule_spec
       -tag tag_name
```

Data Types

rule_name	string
via_rule_spec	specification
tag_name	string

ARGUMENTS

rule_name
Specifies the name of the via rule.

-via_rule via_rule_spec
Specifies the via rule via_rule_spec between strategies, and also between the strategies and existing shapes. The via rule defines the intersection locations and the via at the locations. Note that -via_rule defined in create_pg_mesh_pattern and create_pg_composite_pattern only applies to the intersections of wires in the same pattern, and do not apply to the intersection of wires of different patterns. The -via_rule defined in create_pg_mesh_pattern and create_pg_composite_pattern do not also apply to the intersection of pattern wires and existing wires.

There are three methods to define the intersection locations in the specification via_rule_spec, where the syntax is defined as follows:

```
{intersection : all}      {via_def} |
{intersection : adjacent} {via_def} |
list_of_filter_rules
```

The first method is "{intersection : all}", where intersection is the keyword and all refers to all intersections between any two orthogonal shapes including straps, rings, standard cell rails created by strategies or any existing shapes in any different layers.

The second method is "{intersection : adjacent}", where intersection is the keyword and adjacent refers to all intersections between any two orthogonal shapes only in adjacent layers.

The third method is to define the intersections based on two sets of shapes using list_of_filter_rules, which contains a list of filtering rule. Each filtering rule is specified inside a curly brace pair. The syntax of each filtering rule is as follows:

```
{{set_1}{filter_1}}{{set_2}{filter_2}}{via_def} |
{intersection : undefined}{via_def}
```

where set_1 refers to the first set of shapes and filter_1 refers to the filtering operations for this set. set_2 and filter_2 refer to the second set of shapes and the filtering operations for the second set, respectively. set_1 or set_2 is defined as follows:

```
{strategies : strategy_name_list} |
{existing : shape_type } |
{macro_pins | terminals : all | pin_names }
{tags: {tag_names} }
```

where strategies is the keyword for strategies, existing is the keyword for existing PG wires, macro_pins is the keyword for macro pins and terminals is the keyword for terminals. Strategy names are specified by strategy_name_list. Existing wires are specified by shape type shape_type. Valid shape_type options are: ring, strap, macro_conn, follow_pin and std_conn. The key-

words ring, strap, follow_pin, std_conn, and macro_conn specify ring-type, strap-type, follow pin type, standard cell connection-type, macro pin connection-type and respectively. Macro pins or terminals could be specified by the all keyword, or by a list of pin names pin_names. Tags could be specified by list of tag names. The filtering operations of strategy shapes, existing shapes and tags are defined as follows:

```
{nets : net_list}
{layers : layer_list}
{width : {lower_w upper_w}}
```

nets, layers, width keywords can be used to specify the wire net names by net_list, wire layer names by layer_list or width range by {lower_w upper_w}.

The filtering operations of macro pins are defined as follows:

```
{nets : net_list}
{layers : layer_list}
{width : {lower_w upper_w}}
{height : {lower_h upper_h}}
{width_height_ratio : {lower_r upper_r}}
```

height and width_height_ratio refer to pin height and width height ratio respectively.

By specifying two sets of shapes by strategy names, existing shapes or macro pins, and certain filtering operations by net names, layer names, wire width ranges or types, the via locations are the intersections between these two subsets of shapes. For those intersections which do not belong to any filtering rule, the via definition can be specified by

```
{intersection : undefined}{via_def}
```

where intersection is the keyword, undefined refers to the remaining intersections which do not belong to any filtering rule. By default, vias are only created at the intersections which belong to a filtering rule, unless otherwise specified. Multiple via filtering rules, such as intersection location and via definition, can be defined and separated by curly braces where each filter rule is in one curly brace pair.

With the intersection definition, the via specification {via_def} is in the following format:

```
{via_master : via_master_list}
```

where via_master is the keyword for both via masters and via master rules. via_master_list specifies the list of via masters. Via masters include the via contact code defined in the technology file or user-defined via cells. via_rule_list is a list of PG via rules. Via master rules are defined by set_pg_via_master_rule command. NIL can be used as a keyword to indicate that no via is created. default can be used to describe the default vias in the specified location. For each specified intersection, vias are created based on the specified via masters or via master rules. If NIL is not specified in the list, the default contact code is used to complete a stacked via or to replace a via with DRCs when applicable.

By default, the default via defined in the technology file is created at each intersection of orthogonal wires in different layers.

-tag tag_name
: Specifies a tag name to assign to all vias identified through the associated strategies. This tag name is used as contents of an user attribute tag for filtering collections at later stages. By default, no tags are assigned.

DESCRIPTION

Specifies the via rule between two different strategies or a strategy and existing objects. The via rule can be used with the compile_pg command when creating the power and ground network.

EXAMPLES

The following example specifies a via rule with name r1 where vias will be created only between adjacent layers.

```
prompt> set_pg_strategy_via_rule r1 \
```

```
      -via_rule {{intersection: adjacent}{via_master: default}}
```

The following example specifies a via rule with name r2 which includes a list of filtering rules. Vias VIA23 will be created at the intersection between the shapes from strategy s1 at layer M2 and the shapes from strategy s2 at layer M3. VIA34 will be created at the intersection between the shapes from strategy s2 at layer M3 and all existing ring shapes. Vias will not be created at other intersections between strategy shapes and existing shapes.

```
  prompt> set_pg_strategy_via_rule r2 \
     -via_rule { \
          {{{strategies: s1}{layers: M2}}{{strategies: s2}{layers: M3}} \
                  {via_master: VIA23}} \
          {{strategies: s2}{existing : ring} \
                  {via_master : VIA34}} \
          {{intersection: undefined}{via_master: NIL}} \
     }
```

The following example specifies a via rule with name r3 which includes a list of filtering rules. Vias VIA23 and VIA34 will be created at the intersection between parallel shapes from strategy s1 at layer M2 and the shapes from strategy s2 at layer M4. Vias will not be created at all other intersections.

```
  prompt> set_pg_strategy_via_rule r3 \
       -via_rule { \
          { {{strategies: s1} {layers: M2}} \
            {{strategies: s2} {layers: M4}}{via_master: {VIA23 VIA34}} \
            {between_parallel: true} } \
          { {intersection: undefined}{via_master: NIL} } \
     }
```

The following example specifies a via rule with name r4 which includes a list of filtering rules. Vias VIA23 and VIA34 will be created at the intersection between parallel shapes from strategy s1 at layer M4 and the shapes from existing std rail at layer M2. Vias will not be created at all other intersections.

```
  prompt> set_pg_strategy_via_rule r4 \
       -via_rule { \
          {{{strategies: s1} {layers: M4}}{{existing: std_conn} {layers: M2}} \
            {via_master: {VIA23 VIA34}}{between_parallel: true}} \
           {{intersection: undefined}{via_master: NIL}} \
     }
```

The following example specifies a via rule with name r5 which includes a list of filtering rules. Default vias will be created at the intersection between shapes from strategy s1 and the existing shapes with tag name std_rail. Vias will not be created at all other intersections.

```
  prompt> set_pg_strategy_via_rule r5 \
       -via_rule { \
          {{strategies: s1}{tags: {std_rail}}{via_master: {default}}} \
           {{intersection: undefined}{via_master: NIL}} \
     }
```

The following example specifies a via master "staple_via" with contact code VIA12C, cuts of 2 columns and 1 row, horizontal pitch 0.2. The via rule rail_rule includes a list of filtering rules. Via master staple_via will be created at the intersection between parallel shapes from strategy std_1 and the shapes from existing straps at layer M2. Vias will not be created at all other intersections.

```
  prompt> set_pg_via_master_rule staple_via -contact_code VIA12C \
     -via_array_dimension {2 1} -allow_multiple {0.2 0}
  prompt> create_pg_std_cell_conn_pattern rail_pattern -layers M1
  prompt> set_pg_strategy std_1 -core \
     -pattern {{pattern: rail_pattern} {nets: VDD VSS} }
  prompt> set_pg_strategy_via_rule rail_rule \
     -via_rule { \
        { {strategies: {std_1}} \
          {{existing: strap}{layers: M2}} \
          {via_master:staple_via} {between_parallel: true} } \
        { {intersection: undefined}{via_master: NIL} }}
  prompt> compile_pg -strategies {std_1} -via_rule {rail_rule}
```

MORE EXAMPLES

For more example, please start GUI and invoke the following command.
gui_show_task -task "Design Planning:PG Planning->Examples->Overview"

SEE ALSO

compile_pg(2)
remove_pg_strategy_via_rules(2)
report_pg_strategy_via_rules(2)
set_pg_strategy(2)
set_pg_via_master_rule(2)
